

Supply Chain Integration Hub

A full-stack, multi-tenant purchase order and inventory management platform where two independent organizations — **System 1** and **System 2** — exchange purchase orders, stock offers, approvals, shipment updates, and inventory movements exclusively through **SAP Cloud Platform Integration (SAP CPI)**. Neither side ever calls the other directly; every cross-organization message is routed, validated, format-converted, and mapped by SAP CPI acting as the neutral integration layer between them.

Backend: **Java 21 / Spring Boot 4.1.0 / MongoDB Atlas**. Frontend: **React 19 / Vite**. Integration: **SAP Cloud Platform Integration (5 iFlows)**.

This README documents the system as implemented in this repository, plus the SAP CPI integration flows (iFlows) that sit outside this codebase on the SAP Cloud Integration tenant. Sections describing iFlow internals reflect the documented message contract this backend was built against (see docs/PO_CONTRACT.md) and the iFlow structure as configured on the CPI tenant; the iFlow definitions themselves (Groovy scripts, mappings) are SAP CPI design-time artifacts, not files in this Git repository.

Table of Contents

1. Architecture Overview
2. Key Features
3. Tech Stack
4. Project Structure
5. Getting Started
6. Environment Variables
7. Running the Application
8. Database Design
9. REST API Reference
10. SAP CPI Integration Architecture
11. Order Lifecycle & Status Model
12. Roles & Access Control
13. Known Limitations & Roadmap
14. Testing
15. Security Notes
16. License

Architecture Overview

Two simulated organizations trade purchase orders through one shared integration bridge:

- **System 1** is the human-operated side. Real users log into the React frontend under one of three roles — Vendor, Procurement, Admin— and interact with orders, shipments, inventory, suppliers, and alerts through the UI.
- **System 2** is **not** a second deployed application. It's an in-process simulator (System2ProcurementOrderGenerator and related services) running inside the exact same Spring Boot backend, isolated by its own tenant discriminator (SystemId.SYSTEM2). On a scheduled timer it autonomously raises purchase orders addressed to System 1's vendor, and it autonomously approves or occasionally rejects the purchase orders System 1's procurement side sends out. Architecturally it is not a shortcut: it dispatches through the exact same outbound code path (CpiClient) and the exact same SAP CPI endpoint a real, physically separate System 2 backend would use.

- **SAP CPI** is the only channel between them. Every purchase order, stock offer, approval decision, and inventory movement crosses through one of five integration flows (iFlows), each of which parses a control header, routes/converts the message format, applies message mapping, and forwards it to the destination backend's REST API.

One backend deployment and one MongoDB database serve both tenants. Every persisted document (Order, Alert, Inventory, Shipment, Supplier) carries a SystemId (SYSTEM1 / SYSTEM2) discriminator, and all reads are scoped to the authenticated user's tenant through a single `Tenant` helper class — no endpoint hardcodes which system it's serving.

Component	Connects via	Role
React SPA (System 1 users)	JWT / REST → Spring Boot Backend	Vendor, Procurement, and Admin/Manager login and UI
Spring Boot Backend	Single deployable process	Hosts the REST API for System 1; runs the System 2 simulator and all order-lifecycle schedulers; persists to MongoDB Atlas
System 2	Runs inside the same backend process	Isolated tenant, no UI of its own — simulated counterparty organization
Outbound path	CpiClient → SAP CPI	Backend dispatches purchase orders, approvals, stock offers, and inventory updates out to CPI
Inbound path	SAP CPI → CpiInboundController	CPI delivers routed/converted/mapped messages back into the backend
SAP CPI	External integration layer	iFlow 1 — PO outbound; iFlow 2 — Approval callback; iFlow 3 — Stock offer/rejection; iFlow 4 — Inventory update; iFlow 5 — Low-stock alert email

Key Features

- **Role-based authentication** — JWT-secured login/registration across four roles (Vendor, Procurement, Admin), BCrypt password hashing, and automatic account lockout after repeated failed logins.
- **Dual-lane purchase order negotiation** — a human-operated vendor lane (stock check → buyer response → vendor confirmation) and an autonomous robot-vendor lane (auto-dispatch → simulated accept/reject), both flowing through the same SAP CPI bridge.
- **Multi-format outbound payloads** — every purchase order is dispatched to SAP CPI as JSON, XML, or CSV depending on the order's own format, proving the integration contract is format-agnostic.
- **Resilient CPI bridge** — OAuth2 client-credentials authentication with a cached Bearer token, retry-with-backoff on transient failures, idempotency-key deduplication on inbound messages, and an open-PO cap to protect against message floods.
- **Full audit trail** — every message that crosses the CPI bridge (outbound or inbound, success or failure) is recorded to a searchable `integration_logs` collection, including a correlation-ID trace view that reconstructs a purchase order's entire cross-system journey.
- **Tenant isolation on shared infrastructure** — one backend, one database, two logically isolated organizations, enforced centrally through a single `Tenant` helper rather than scattered conditionals.

- **Inventory management with low-stock alerting** — stock receive/sell operations, a per-item reorder threshold, automatic alert generation, and an outbound low-stock email flow via SAP CPI.
- **Shipment lifecycle automation** — confirmed orders progress through Processing → In Transit → Delivered on scheduled background jobs, with dedicated "recovery" schedulers that catch and advance any order that appears stuck.
- **Cross-tenant admin oversight** — a dedicated Admin/Manager view aggregates order and inventory analytics across both tenants, plus full user administration (lock/unlock, role changes).

Tech Stack

Backend

Category	Technology
Language	Java 21
Framework	Spring Boot 4.1.0
Web layer	Spring Web MVC (spring-boot-starter-webmvc)
Security	Spring Security (stateless, JWT-based)
Authentication	JWT (jjwt-api / jjwt-impl / jjwt-jackson 0.12.6)
Database	MongoDB Atlas via Spring Data MongoDB
Validation	Jakarta Bean Validation (spring-boot-starter-validation)
Boilerplate reduction	Lombok
Build tool	Apache Maven (via the Maven Wrapper — no local Maven install required)
Hot reload	Spring Boot DevTools
Testing	spring-boot-starter-test, spring-security-test

Frontend

Category	Technology
Library	React 19
Build tool	Vite 8
Routing	React Router DOM v7
HTTP client	Axios
State management	React hooks (useState/useEffect/useCallback) + Context API for auth
Linting	ESLint (flat config) with React Hooks / React Refresh plugins

Integration

Category	Technology
iPaaS	SAP Cloud Platform Integration (SAP CPI) — 5 iFlows
Outbound auth	OAuth2 client-credentials grant
Inbound auth	Shared-secret header (X-CPI-Secret)

Project Structure

Backend (integration-hub/)

Path	Description
pom.xml	Maven project descriptor
mvnw, mvnw.cmd	Maven Wrapper — builds/runs without a local Maven install
docs/PO_CONTRACT.md	Canonical cross-system message contract
docs/PROJECT_CHECKPOINT.md	Internal build/status log
.env.example	Environment variable template — copy to .env, fill in real values
src/main/java/com/supplychain/integration_hub/*Controller.java	REST endpoints (21 files)
src/main/java/com/supplychain/integration_hub/*Service.java	Business logic (21 files)
src/main/java/com/supplychain/integration_hub/*Repository.java	Spring Data MongoDB interfaces (8 files)
src/main/java/com/supplychain/integration_hub/*Scheduler.java	@Scheduled background jobs (6 files)
src/main/java/com/supplychain/integration_hub/(remaining files)	Models, DTOs, enums, security config, exception handling
src/main/resources/application.properties	Central Spring configuration

Frontend

Path	Description
package.json, vite.config.js, index.html	Project and build configuration
src/main.jsx, src/App.jsx	Routing, per-role navigation, top-level layout
src/api/axios.js	Shared Axios instance + JWT interceptor
src/api/services.js	Named endpoint wrappers (partial coverage)
src/context/AuthContext.jsx	Global auth state (token / role / systemId)
src/components/Pagination.jsx	Shared pagination control
src/pages/Login.jsx, Dashboard.jsx, VendorDashboard.jsx, ProcurementDashboard.jsx	Role dashboards
src/pages/Orders.jsx, Shipments.jsx, Inventory.jsx, Suppliers.jsx, Alerts.jsx	Core feature pages
src/pages/UserManagement.jsx, IntegrationLogs.jsx	Admin/oversight pages
src/pages/AdminVendorView.jsx, AdminProcurementView.jsx	Cross-tenant oversight views

Getting Started

Prerequisites

- **JDK 21**
- **Node.js 18+** and npm
- A **MongoDB Atlas** cluster (or any MongoDB 6+ instance)
- An **SAP Cloud Platform Integration** tenant with the five iFlows described below deployed and active
- (Development only) **ngrok** or an equivalent tunnel, to expose your local backend to SAP CPI's approval-callback iFlow

Clone and configure

```
git clone <this-repository-url>
cd integration-hub
cp .env.example .env
# edit .env with your own MongoDB URI, JWT secret, and CPI credentials
```

Backend setup

```
cd integration-hub
./mvnw clean install
./mvnw spring-boot:run
```

The API starts on `http://localhost:8080`.

Frontend setup

```
cd frontend
npm install
npm run dev
```

The frontend starts on Vite's default dev port (typically `http://localhost:5173`) and expects the backend at `http://localhost:8080/api` (currently hardcoded in `src/api/axios.js` — see Known Limitations).

Environment Variables

All secrets are supplied via environment variables, loaded from a gitignored `.env` file at the backend root (see `.env.example` for the template). **Never commit real values** — only variable names are documented here.

Variable	Required	Purpose
MONGODB_URI	Yes	MongoDB Atlas connection string
APP_JWT_SECRET	Yes	HMAC signing key for JWTs (generate with <code>openssl rand -hex 32</code>)
CPI_BASE_URL	Yes	Base URL of your SAP CPI tenant
CPI_TOKEN_URL	Yes	OAuth2 token endpoint used for the client-credentials grant
CPI_CLIENT_ID	Yes	OAuth2 client ID for CPI
CPI_CLIENT_SECRET	Yes	OAuth2 client secret for CPI
CPI_INBOUND_SECRET	Recommended	Shared secret CPI must send as X-CPI-Secret on inbound calls; if blank, the check is skipped (dev-only)
CPI_IFLOW3_STOCK_OFFER_U	Yes (for Lane A stock-offer flow)	Direct endpoint URL for the

Variable	Required	Purpose
RL		iFlow 3 receiver
CPI_IFLOW4_INVENTORY_UPDATE_URL	Yes (for inventory sync)	Direct endpoint URL for the iFlow 4 receiver
CPI_IFLOW5_ALERT_EMAIL_URL	Yes (for low-stock email)	Direct endpoint URL for the iFlow 5 receiver

Additional non-secret configuration (order lifecycle timer delays, retry/backoff policy, open-PO caps, simulator interval) lives directly in `application.properties` and can be tuned without touching environment variables.

Running the Application

1. Start the backend: `./mvnw spring-boot:run` (from `integration-hub/`).
2. Start the frontend: `npm run dev` (from `frontend/`).
3. Open the frontend URL, register an account under one of the three role tabs (Vendor, Procurement, Admin — Admin also serves the Manager role), and log in.
4. Purchase orders raised by System 1 Procurement against a System 2 vendor dispatch automatically to SAP CPI on creation. System 2's own activity (raising orders against System 1's vendor, approving/rejecting System 1's outgoing orders) runs on its own schedule (`simulator.interval-ms`) once the backend is up — no separate process to start.
5. Admin/Manager users can pause, resume, or manually trigger one iteration of the System 2 simulator via `GET/POST /api/simulator/status|start|stop|fire`.

Database Design

MongoDB (document store, no relational joins) via Spring Data MongoDB. Relationships between collections are maintained at the application layer using reference fields, not foreign keys.

Collection	Backing Class	Purpose
orders	Order	Purchase orders, both directions, both tenants, including cross-tenant mirror records
users	User	Application accounts and roles
suppliers	Supplier	Per-tenant supplier directory
shipments	Shipment	Physical shipment records linked to orders
inventory	InventoryItem	Per-tenant stock items
alerts	Alert	Role-targeted system notifications
integration_logs	IntegrationLog	Full audit trail of every message that crossed the CPI bridge

Key fields on ``orders``: `orderId` (human-readable ID), `direction` (INBOUND/OUTBOUND), `status` (internal fulfilment lifecycle — see below), `poStatus` (cross-system contract state — see below), `correlationId` (immutable, reconciles a PO with its later approval callback), `idempotencyKey` (write-once inbound dedup key), `sourceSystem/targetSystem` (wire-level routing addresses), `systemId` (tenant discriminator), `format` (json/xml/csv).

Indexes: explicit indexes are declared on `User.email` (unique), and on `IntegrationLog.timestamp`, `IntegrationLog.correlationId`, `IntegrationLog.eventType`, and `IntegrationLog.status` — the fields the log-search and correlation-trace features query most heavily.

REST API Reference

All endpoints require Authorization: Bearer <JWT> unless noted otherwise, and are further restricted by role (see Roles & Access Control).

Authentication

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate and receive a JWT
POST	/api/auth/register	Create an account and receive a JWT

Orders

Method	Endpoint	Purpose
GET	/api/orders	List orders visible to the caller's role/tenant
GET	/api/orders/paged	Paginated, filterable, searchable, sortable order list
GET	/api/orders/active /past /pending-approvals /buyer-decisions	Scoped order views
GET	/api/orders/{id} /{id}/availability	Single order / per-line-item stock availability
POST	/api/orders	Create an order
PUT	/api/orders/{id}/notify-stock /respond-stock	Vendor stock-check negotiation (Lane A)
POST	/api/orders/{id}/send	Manually dispatch an order to CPI (iFlow 1)
PUT	/api/orders/{id}/send-offer / POST /{id}/stock-offer	Vendor stock offer via CPI iFlow 3 (send / receive)
PUT	/api/orders/{id}/reject /confirm-supply /cancel	Order decision endpoints
POST	/api/orders/{id}/vendor-confirm-supply /cancel-vendor	Vendor-side finalization

CPI Inbound Bridge

Method	Endpoint	Auth	Purpose
POST	/api/cpi/inbound/po	X-CPI-Secret (no JWT)	Receive a purchase order via iFlow 1
POST	/api/cpi/inbound/stock-offer	X-CPI-Secret (no JWT)	Receive a vendor stock decision via iFlow 3
POST	/api/cpi/inbound/inventory-update	X-CPI-Secret (no JWT)	Receive an inventory movement via iFlow 4
POST	/api/cpi/inbound/approval	X-CPI-Secret (no JWT)	Receive an approval/rejection callback via iFlow 2

Inventory, Shipments, Suppliers, Alerts

Resource	Key endpoints
Inventory	GET/POST /api/inventory, POST /receive, POST /sell, GET /alerts, GET /low-stock, PUT /{id}/quantity
Shipments	GET /api/shipments (+ /active, /past), POST /api/shipments, PUT /{id}/status, PUT /{id}/cancel
Suppliers	GET/POST/PUT/DELETE /api/suppliers, PUT /{id}/status
Alerts	GET /api/alerts (+ /active), PUT /{id}/resolve, PUT /resolve-all, POST /generate

Dashboards, Admin, Logs

Resource	Key endpoints
Dashboards	GET /api/dashboard/stats, /api/dashboard/vendor, /api/dashboard/procurement
Admin	GET /api/admin/summary, /api/admin/orders, /api/admin/inventory-analysis
Users	GET/PUT/DELETE /api/users/** (lock/unlock, role changes)
Integration Logs	GET /api/logs, /api/logs/{correlationId}, /api/logs/status/{status}, plus the richer /api/integration-logs/** search surface
Simulator	GET /api/simulator/status, POST /start /stop /fire

SAP CPI Integration Architecture

Every cross-organization message passes through SAP Cloud Platform Integration as five separate integration flows (iFlows). **The iFlows themselves are SAP CPI design-time artifacts, configured and deployed on the SAP CPI tenant — they are not files inside this Git repository.** The breakdown below documents each iFlow's components and the responsibility of each Groovy script, based on the message contract this backend was built and tested against (docs/PO_CONTRACT.md) and the step-by-step iFlow structure as designed on the tenant. Where the internal step-by-step composition of an iFlow was not documented to the same level of granularity as iFlow 1 and iFlow 2, that is stated explicitly below rather than guessed at.

iFlow overview

iFlow	Trigger / Sender	Purpose	Backend endpoint it ultimately calls
iFlow 1 — PO Outbound	HTTPS Sender, called by CpiClient.sendPo	Routes a newly raised purchase order (JSON, XML, or CSV) to its destination tenant	POST /api/cpi/inbound/po
iFlow 2 — Approval Callback	HTTPS Sender, /approval-callback	Carries a vendor's approve/reject decision back to the originating	POST /api/cpi/inbound/approval

iFlow	Trigger / Sender	Purpose	Backend endpoint it ultimately calls
		procurement side	
iFlow 3 — Stock Notification	HTTPS Sender, /stock-offer (and a dedicated direct-URL variant)	Carries a vendor's stock offer or rejection to the counterparty procurement	POST /api/cpi/inbound/stock-offer
iFlow 4 — Inventory Update	HTTPS Sender, direct URL	Carries a physical stock movement (vendor supplied goods / procurement received goods)	POST /api/cpi/inbound/inventory-update
iFlow 5 — Low-Stock Alert Email	HTTPS Sender, direct URL	Carries a consolidated low-stock batch (posted once every 10 minutes by the backend) and sends it out as an email	Email adapter (no backend callback)

The #meta control line — the contract every Groovy script parses

Outbound bodies for iFlow 1 (and, in an adapted form, the other iFlows) begin with a single plaintext control line before the actual payload, because the SAP CPI trial tenant used for this project strips inbound HTTP headers and query parameters — so routing information has to travel *inside* the body itself*:

```
#meta source=system1 target=system2 correlationId=PO-20260618-0001 idempotencyKey=PO-20260618-0001
format=json
<payload in json | xml | csv>
```

- **source / target** — system1 or system2; tells CPI which tenant's inbound endpoint to call.
- **correlationId** — immutable; survives every transform; the approval callback in iFlow 2 must echo it back unchanged so the originating order can be reconciled.
- **idempotencyKey** — a write-once dedup key the receiving backend uses to guard against duplicate delivery.
- **format** — json, xml, or csv; tells the Router which converter branch to use.

Every Groovy script described below that's labelled "parses the control line" is doing the same fundamental job: read this first line, extract these properties onto the CPI message exchange, and strip the line from the body so the remaining converter/mapping steps only ever see the actual payload.

iFlow 1 — Purchase Order Outbound (in full detail)

This is the primary iFlow and the one whose component chain is documented here in the most detail:

```
HTTPS Sender
↓
Groovy Script — ParseControlParams
↓
Router
↓
CSV-to-XML Converter | JSON-to-XML Converter (branch selected by the "format" property)
↓
Message Mapping
↓
XML-to-JSON Converter
↓
Groovy Script — (post-mapping finalization)
↓
HTTPS Receiver → destination backend's /api/cpi/inbound/po
```

Component-by-component:

- **HTTPS Sender.** The public entry point `CpiClient.sendPo` posts to (`cpi.po-outbound-path=/http/po-outbound`). Configured with CSRF disabled (senders on this trial tenant don't need it) and OAuth2 client-credentials as the inbound security mechanism, matching what `CpiClient` authenticates with.
- **Groovy Script — `ParseControlParams`.** The first processing step. Its job is to read the `#meta` line described above off the top of the raw message body, split it into its `source` / `target` / `correlationId` / `idempotencyKey` / `format` fields, write each of them onto the CPI message as exchange properties (so every downstream step — Router, Mapping, the final Groovy script — can read them without re-parsing text), and then remove that first line from the body entirely. Everything after this step operates on a clean, format-pure payload with no control-line prefix.
- **Router.** Branches purely on the `format` exchange property set by `ParseControlParams` — `json` skips straight to Message Mapping (already close to canonical shape), `xml` and `csv` are each routed to their own converter first.
- **CSV-to-XML Converter / JSON-to-XML Converter.** Standard SAP CPI content-modification steps (not custom Groovy) that normalize the three possible wire formats — `{"order":{...}}` JSON with no wrapper, `<orders><order>...</order></orders>` XML with a wrapper, or a CSV header+row pair — into one common intermediate XML shape that Message Mapping can consume uniformly, regardless of which format System 1 or System 2's backend originally chose for this particular order.
- **Message Mapping.** Transforms the intermediate XML into the canonical purchase-order structure the destination system's inbound API actually expects (`InboundPoRequest`: `poNumber`, `correlationId`, `sourceSystem/targetSystem`, `format`, `counterpartyId/counterpartyName`, line items, `totalAmount`). This is the step that means System 1 and System 2 never need to agree on wire-level field names — only on the canonical contract at this mapping boundary.
- **XML-to-JSON Converter.** The destination backend's `/api/cpi/inbound/po` endpoint expects a JSON request body, so the mapped canonical XML is converted to JSON immediately before delivery.
- **Groovy Script — post-mapping finalization.** The final scripting step before the HTTPS Receiver call. Its responsibility is to re-attach what the mapping/conversion steps don't carry as body content but the destination backend's security layer requires: specifically the `X-CPI-Secret` shared-secret header that `CpiInboundController/SecurityConfig` expects on every `/api/cpi/inbound/**` call (these endpoints are deliberately `permitAll()` at the Spring Security layer and are instead gated by this header check inside the controller, since CPI carries no end-user JWT). It also re-applies the `correlationId` and `idempotencyKey` exchange properties captured back in `ParseControlParams` onto the outgoing JSON body/headers, so the receiving backend can deduplicate and reconcile correctly.
- **HTTPS Receiver.** Delivers the finished JSON body, with the `X-CPI-Secret` header attached, to the destination tenant's POST `/api/cpi/inbound/po`.

iFlow 2 — Approval Callback

```

HTTPS Sender (/approval-callback)
↓
Groovy Script — ParseControlParams
↓
Content Modifier — ValidateApproval
↓
Content Modifier — SetBackendHeaders
↓
HTTPS Receiver → destination backend's /api/cpi/inbound/approval

```

- **HTTPS Sender (`/approval-callback`).** Entry point for a vendor's approve/reject decision travelling back to the side that originally raised the PO.
- **Groovy Script — `ParseControlParams`.** Same responsibility as in iFlow 1: extracts `source/target/correlationId` from the control line so the callback can be routed to the correct

originating tenant and reconciled against the correct order — this is the step that makes the "immutable correlationId" guarantee in docs/PO_CONTRACT.md actually work end to end.

- **`ValidateApproval`**. Validates that the incoming decision payload is well-formed (a decision of APPROVED or REJECTED, a non-empty correlationId/poNumber) before it's allowed to reach the backend — protecting the destination's receiveApproval logic from malformed callbacks.
- **`SetBackendHeaders`**. Attaches the X-CPI-Secret header, sourced from a value stored in SAP CPI's own Security Material (not hardcoded in the iFlow) rather than in application code — keeping that secret out of both the CPI design-time artifact and this repository.
- **HTTPS Receiver**. Delivers the approval/rejection payload to POST /api/cpi/inbound/approval on the originating backend. In this project's development setup, that receiver address is exposed via an ngrok tunnel to the developer's local machine rather than a permanently deployed public endpoint (see Known Limitations).

iFlow 3 — Stock Notification

Carries a vendor's stock offer (a specific quantity they can supply) or an outright rejection from the vendor side back to the counterparty's procurement side, landing on POST /api/cpi/inbound/stock-offer. Two backend paths feed this iFlow: a shared endpoint path (cpi.stock-offer-path=/http/stock-offer) and a dedicated per-environment direct URL (cpi.iflow3-stock-offer-url) used specifically for the System1-vendor-to-System2-procurement lane. The payload contract mirrors iFlow 1's general shape (a correlationId-keyed JSON body carrying decision, offeredQuantity, requiredQuantity, and an optional note) but this iFlow's exact internal step-by-step composition (whether it uses its own dedicated Groovy scripts or reuses shared ones) was not documented to the same granularity as iFlow 1/iFlow 2 during this project's build, so it is described here at the contract level rather than the component level.

iFlow 4 — Inventory Update

Carries a physical stock movement — VENDOR_SUPPLY (vendor confirmed supply, decreasing System 1's stock) or PROCUREMENT_RECEIVE (goods received, increasing System 1's stock) — to POST /api/cpi/inbound/inventory-update, fed by a dedicated direct URL (cpi.iflow4-inventory-update-url). One notable contract detail baked into the backend: this endpoint's schema only accepts correlationId, orderId, eventType, sku, itemName, quantity, unit, and reason — sourceSystem/targetSystem are deliberately **not** included, because iFlow 4's XSD validation (an xs:all group) rejects the payload if they're present. As with iFlow 3, this iFlow's precise internal component chain wasn't documented at the same depth as iFlow 1/2 and is described here by contract rather than by step.

iFlow 5 — Low-Stock Alert Email

The backend's InventoryAlertScheduler posts one consolidated batch of currently-active low-stock alerts to this iFlow every 10 minutes (cpi.iflow5-alert-email-url), and the iFlow's job is to turn that batch into an outbound email to the configured recipients — this is the one iFlow in the set that does not call back into either backend; it terminates in an email/notification adapter instead of an HTTPS receiver.

Order Lifecycle & Status Model

Order documents move through **two parallel status vocabularies**, deliberately kept separate:

- **`OrderStatus`** (internal fulfilment lifecycle): REQUESTED → STOCK_NOTIFIED/BUYER_APPROVED → CONFIRMED → PROCESSING → IN_TRANSIT → DELIVERED, with REJECTED/CANCELLED/VENDOR_REJECTED/BUYER_REJECTED off-ramps. Advanced partly by explicit user action and partly by scheduled jobs that move CONFIRMED orders through PROCESSING → IN_TRANSIT → DELIVERED on fixed timers.
- **`PoStatus`** (the cross-system contract both System 1 and System 2 obey): DRAFT → SENT → RECEIVED → APPROVED|REJECTED → FULFILLED → SHIPPED (+ FAILED for a retryable dispatch failure). This is the vocabulary that actually crosses the CPI bridge and is set independently of OrderStatus.

Because System 1 and System 2 share one backend and one database, several services maintain **mirror** Order documents: whenever a System 1 order's status changes, a same-orderId/correlationId counterpart document tagged for System 2 (direction flipped) is created or updated, so a query scoped to System 2's tenant sees the same order from System 2's point of view without ever touching System 1's actual documents directly.

Roles & Access Control

Role	Scope
Vendor	Own tenant's outbound sales orders, shipments, inventory, alerts
Procurement	Own tenant's inbound purchase orders, suppliers, shipments, inventory, alerts
Admin	Cross-tenant oversight (both Vendor and Procurement views), full user administration, integration logs, simulator controls
Manager	Same cross-tenant oversight as Admin, served by the same login tab

Authorization is enforced centrally in a single Spring Security filter chain configuration — every resource prefix (`/api/orders/**`, `/api/inventory/**`, `/api/admin/**`, etc.) has one explicit `hasAnyAuthority(...)` rule, rather than authorization logic scattered across individual controllers. CPI-facing endpoints (`/api/cpi/**`) are the one exception: they're `permitAll()` at the Spring Security layer because SAP CPI carries no end-user JWT, and are instead gated by the X-CPI-Secret header check described in the iFlow breakdown above.

Known Limitations & Roadmap

Documented transparently rather than glossed over:

- **Multi-line-item purchase orders are not yet fully end-to-end.** The outbound iFlow 1 payload currently carries only the first line item of a multi-item order; full line-item support through the mapping layer is a planned enhancement.
- **Tenant-scoping is not yet fully consistent** across all secondary collections — `Shipment`, `Alert`, `Supplier`, and `Inventory` are not `systemId`-scoped as thoroughly as `Order`, so some admin-facing counts are effectively global rather than strictly per-tenant.
- **The frontend's API base URL is currently hardcoded** (`http://localhost:8080/api` in `src/api/axios.js`) rather than read from an environment variable (e.g. `VITE_API_URL`).
- **CORS is currently permissive** (`allowedOriginPatterns: "*" with credentials enabled`) for development convenience and should be locked down to an explicit origin allow-list before any production exposure.
- **The iFlow 2 approval-callback receiver currently depends on an ngrok tunnel** to reach the developer's local backend rather than a permanently deployed public endpoint.
- **No automated test suite yet.** `spring-boot-starter-test` and `spring-security-test` are present as dependencies, but the project has been verified through manual, scenario-driven testing to date; unit/slice tests are a planned addition.
- **A duplicate, stale copy of the frontend scaffold** exists in the repository and is pending removal.
- **Tailwind CSS and Recharts are installed as dependencies but not currently wired into the build/UI** — either integrate them or remove them as dead weight.

Planned enhancements: full multi-line-item support, complete tenant-scoping of secondary collections, an ML-based order risk-scoring iFlow (an `Order.riskScore` field already exists in the schema, ready to

be populated), an end-of-day summary report iFlow, containerized deployment with CI, and a permanent public deployment replacing the ngrok-based development setup.

Testing

Testing to date has been manual and scenario-driven: full end-to-end verification of both negotiation lanes (human-operated vendor and autonomous robot vendor), all three outbound wire formats (JSON/XML/CSV), inbound idempotency deduplication, the open-PO cap, and the low-stock alert flow. A developer-only utility endpoint (POST /api/cpi/test/send?format=json|xml|csv) exists specifically to fire a test purchase order directly at SAP CPI without going through the UI, useful for isolating whether an issue is in the application or in the iFlow configuration itself. Automated unit and integration tests are on the roadmap (see above).

Security Notes

- All secrets (MONGODB_URI, APP_JWT_SECRET, CPI_CLIENT_ID/CPI_CLIENT_SECRET, CPI_INBOUND_SECRET) are supplied via environment variables and must never be committed to source control.
- Passwords are hashed with BCrypt; accounts lock automatically after 5 consecutive failed login attempts.
- JWTs are signed with HMAC-SHA and carry role, entityId, and systemId claims; sessions are stateless.
- CPI-facing inbound endpoints are intentionally excluded from JWT processing and are instead gated by a shared-secret header (X-CPI-Secret), since SAP CPI's receiver adapter carries no end-user token.
- **Before pushing this repository publicly**, review .env.example and application.properties for any environment-specific values (cluster hostnames, recipient email addresses, prior credential references) that should be genericized or removed — see the note from the assistant accompanying this README for specifics found during this review.